Assignment 5

Name: Tanishq Tajne

PRN: 123B1B276

1. Introduction

The **College Event Attendance System** is a full-stack web application developed to digitize the process of recording student attendance during college events. Traditionally, attendance is recorded manually on paper, which can lead to errors, requires significant time to manage, and makes record storage and retrieval difficult. This project replaces the manual process with a web-based system where students can register their attendance through an online form. The system automatically stores and manages attendance data, improving accuracy, accessibility, and efficiency.

THEORY

2. Technology Stack

2.1 Node.js

Node.js is an open-source, cross-platform JavaScript runtime environment built on Chrome's **V8 JavaScript engine**. It allows JavaScript to be executed on the server side, enabling developers to use the same language for both frontend and backend development. Node.js uses an **event-driven, non-blocking I/O model**, which makes it lightweight and efficient for handling multiple requests simultaneously. In this project, Node.js acts as the runtime environment that executes the backend server.

2.2 Express.js

Express.js is a minimal and flexible web framework built on top of Node.js. It simplifies backend development by providing features for routing, middleware handling, and HTTP request management. Express enables developers to easily create REST APIs using HTTP methods such as **GET, POST, PUT, and DELETE**. In this project, Express.js manages API routes, processes incoming requests, parses JSON data using express.json(), and serves as the primary framework for backend logic.

2.3 MongoDB (NoSQL Database)

MongoDB is a **document-oriented NoSQL database** that stores data in flexible, JSON-like documents called **BSON (Binary JSON)**. Unlike traditional relational

databases that require fixed schemas, MongoDB allows flexible data structures, making it suitable for applications where data requirements may evolve over time.

Key characteristics of MongoDB include:

- **Document Model:** Data is stored as documents inside collections, similar to rows inside tables in relational databases.
- **Schema Flexibility:** Each document can have a different structure, allowing adaptable data storage.
- **Scalability:** MongoDB supports horizontal scaling through sharding.
- **Indexing:** Secondary indexes improve query performance and data retrieval speed.

In this project, MongoDB stores attendance records in the **attendances collection** within the **college_attendance database**.

2.4 Mongoose (ODM)

Mongoose is an **Object Data Modeling (ODM)** library for MongoDB and Node.js. It provides a structured way to define schemas and interact with MongoDB collections. Although MongoDB is schema-less, Mongoose allows developers to enforce structure by defining data types, validation rules, and default values.

Mongoose features include:

- Schema definition for structured data modeling
- Data validation and type casting
- Query building and middleware hooks
- Simplified interaction with MongoDB collections

In this project, Mongoose defines the **Attendance schema** with required fields such as studentName, prn, studentClass, eventName, eventDate, and eventTime.

2.5 React.js

React.js is a **component-based JavaScript library** used for building dynamic user interfaces. Developed by Facebook, React allows developers to create reusable UI components and manage application state efficiently.

Key concepts used in this project include:

- **Components:** The interface is divided into reusable components such as AttendanceForm and AttendanceList.
- **State Management:** The useState hook manages form input values and attendance data.

- **Side Effects:** The useEffect hook performs operations such as fetching data from the backend API when the component loads.
- **Virtual DOM:** React uses a virtual representation of the DOM to efficiently update only the necessary parts of the user interface, improving performance.

2.6 Axios

Axios is a **promise-based HTTP client** used for sending asynchronous HTTP requests from the frontend to the backend API. Compared to the native fetch API, Axios provides built-in features such as automatic JSON transformation, request and response interception, and improved error handling.

In this project, Axios is configured with a base URL pointing to the backend server (http://localhost:5000/api) and is used to send requests for retrieving, creating, and deleting attendance records.

3. Architecture

The system follows a **three-tier architecture**, which separates the application into different layers for better maintainability and scalability.

- **Presentation Layer (React):** Handles user interaction, form input, and displays attendance records in a structured format such as a table.
- **Business Logic Layer (Express.js):** Processes requests from the frontend, validates input data, applies business rules (such as duplicate prevention), and communicates with the database.
- **Data Layer (MongoDB):** Stores and manages attendance records as documents inside collections.

4. REST API Design

The backend provides a **RESTful API** to manage attendance data. REST (Representational State Transfer) is an architectural style that uses standard HTTP methods to perform operations on resources.

Method	Endpoint	Description
GET	/api/attendance	Retrieve all attendance records
POST	/api/attendance	Submit a new attendance record
DELETE	/api/attendance/:id	Delete a specific attendance record

Each endpoint represents a resource (attendance records), while the HTTP method defines the action to be performed.

5. CORS (Cross-Origin Resource Sharing)

CORS is a browser security mechanism based on the **Same-Origin Policy**, which prevents web pages from making requests to a different domain, protocol, or port than the one from which they were loaded.

In this project, the **React frontend runs on port 3000**, while the **Express backend runs on port 5000**. Since these ports represent different origins, the browser blocks requests by default. The backend uses the **CORS middleware** to include the Access-Control-Allow-Origin header in responses, allowing the frontend to communicate with the backend.

6. ES6 Modules

The backend uses **ES6 module syntax** (import and export) instead of the older **CommonJS syntax** (require and module.exports). ES6 modules are the modern standard for structuring JavaScript code into reusable modules.

Advantages of ES6 modules include:

- **Static analysis:** Imports are resolved at compile time, enabling better optimization and tooling.
- **Explicit exports:** Only explicitly exported elements can be accessed by other modules.
- **Consistency:** The same module system works in both browser and server environments.

This module system is enabled by setting "type": "module" in the package.json file.

7. Data Model

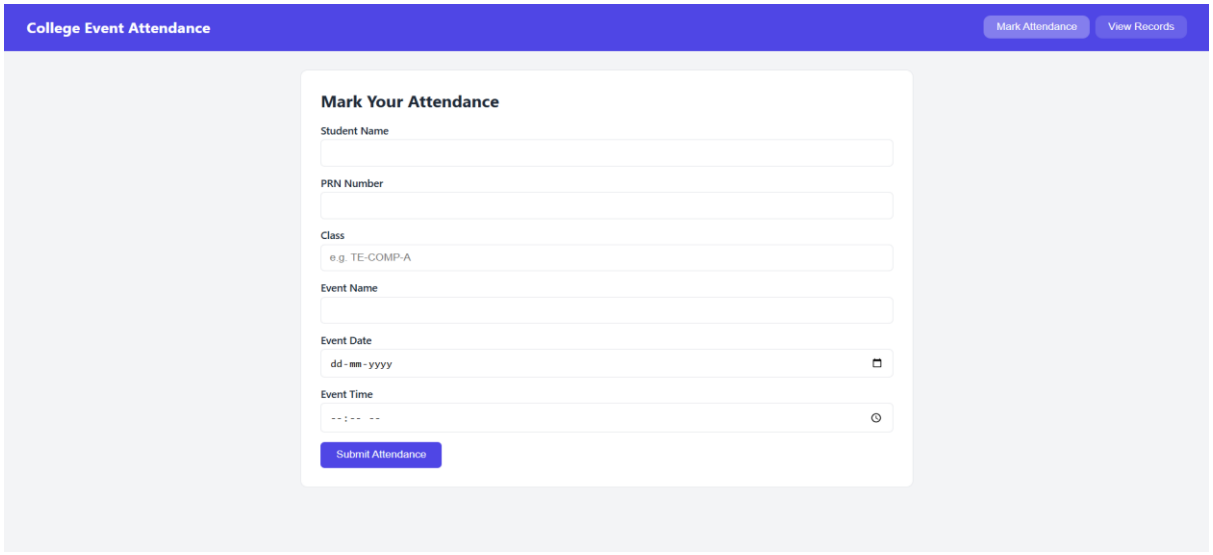
The application stores attendance information using a structured schema defined through Mongoose. Each attendance record contains details such as the student’s name, PRN number, class, event name, event date, and event time. This schema ensures that all required information is consistently stored and validated before being inserted into the database.

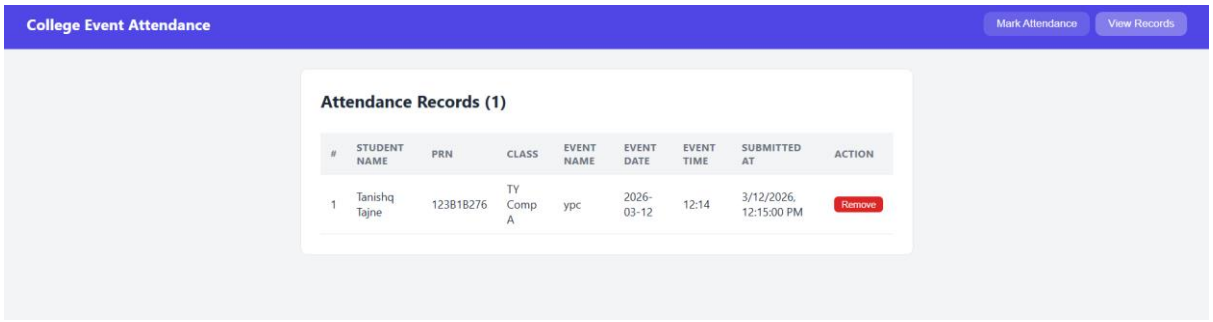
8. Duplicate Prevention

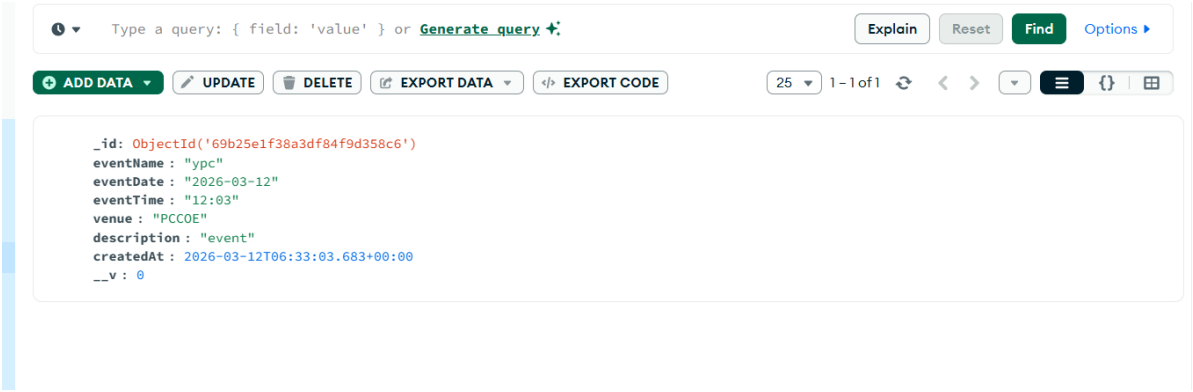
To maintain data integrity, the system enforces a rule that **a student cannot mark attendance more than once for the same event on the same date**. Before inserting a new record, the backend checks the database for an existing document with the same **PRN, event name, and event date**. If a matching record is found, the system returns an HTTP **409 Conflict** response and prevents the duplicate entry from being saved.

9. Conclusion

The College Event Attendance System demonstrates the implementation of a modern **full-stack web application** using React, Node.js, Express.js, and MongoDB. The project highlights key web development principles such as component-based UI design, RESTful API architecture, NoSQL data modeling, cross-origin communication, and modular JavaScript development. By digitizing the attendance process, the system improves efficiency, reduces errors, and provides a scalable solution for managing event participation records.






Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
UPDATE
DELETE
EXPORT DATA
EXPORT CODE
25
1 - 1 of 1
_id: ObjectId('69b25e1f38a3df84f9d358c6')
eventName : "ypc"
eventDate : "2026-03-12"
eventTime : "12:03"
venue : "PCCOE"
description : "event"
createdAt : 2026-03-12T06:33:03.683+00:00
__v : 0